

ASP.NET Core MVC ile C# Web Geliştirme

Kapsamlı Referans Belgesi

OOP · SOLID · MVC · EF Core · DI · LINQ · AJAX

ASP.NET Core MVC ile C# Web Geliştirme — Referans Belgesi

ORTAM

- **Dil ve sürüm:** C# 12 / .NET 8 (ASP.NET Core MVC)
- **Çalıştırma ortamı:** Visual Studio 2022, IIS Express / Kestrel, Package Manager Console
- **Ana kütüphaneler:**
 - `Microsoft.EntityFrameworkCore` (ORM)
 - `Microsoft.EntityFrameworkCore.SqlServer` (SQL Server sağlayıcısı)
 - `Microsoft.EntityFrameworkCore.Tools` (Migration komutları)
- Bootstrap 5 + Bootswatch (UI teması)
- Bootstrap Icons (ikon seti)
- jQuery (DOM manipülasyonu ve AJAX)
- Toastr.js (bildirim kütüphanesi)
- **İşletim sistemi varsayımı:** Windows (Trusted Connection ile SQL Server)
- **Veritabanı:** Microsoft SQL Server (Local DB veya tam sürüm), Trusted Connection ile bağlanır.

PROJE YAPISI

İki ayrı proje belge boyunca referans alınmıştır. Yapılar aşağıdadır.

Kategori Yönetim Projesi (Bulky)

```
BulkyBookWeb/  
├─ Controllers/  
│   ├─ HomeController.cs  
│   └─ CategoryController.cs  
├─ Data/  
│   └─ ApplicationDbContext.cs  
├─ Models/  
│   ├─ Category.cs  
│   └─ ErrorViewModel.cs  
├─ Views/  
│   ├─ Home/  
│   │   ├─ Index.cshtml  
│   │   └─ Privacy.cshtml  
│   ├─ Category/  
│   │   ├─ Index.cshtml  
│   │   ├─ Create.cshtml  
│   │   ├─ Edit.cshtml  
│   │   └─ Delete.cshtml  
│   └─ Shared/  
│       ├─ _Layout.cshtml  
│       ├─ _Notification.cshtml  
│       ├─ _ValidationScriptsPartial.cshtml  
│       └─ Error.cshtml  
│   └─ _ViewImports.cshtml  
│   └─ _ViewStart.cshtml  
├─ Migrations/  
│   └─ (oluşturulan migration dosyaları)  
├─ wwwroot/  
│   ├─ css/  
│   │   ├─ site.css  
│   │   └─ bootswatch-theme.css  
│   ├─ js/  
│   │   └─ site.js  
│   └─ lib/  
│       ├─ bootstrap/  
│       ├─ jquery/  
│       └─ jquery-validation/  
├─ appsettings.json  
└─ Program.cs
```

Hastane Randevu Projesi

```
HastaneYonetim/
├── Controllers/
│   ├── HomeController.cs
│   ├── HastaController.cs
│   ├── DoktorController.cs
│   ├── BransController.cs
│   └── RandevuController.cs
├── Data/
│   └── HastaneDbContext.cs
├── Models/
│   ├── Hasta.cs
│   ├── Doktor.cs
│   ├── Brans.cs
│   └── Randevu.cs
├── DTOs/
│   ├── DoktorDTO.cs
│   └── BransModelDTO.cs
├── ViewModels/
│   └── RandevuViewModel.cs
├── Views/
│   ├── Home/
│   ├── Hasta/
│   ├── Doktor/
│   ├── Brans/
│   ├── Randevu/
│   ├── Shared/
│   ├── _ViewImports.cshtml
│   └── _ViewStart.cshtml
├── wwwroot/
├── appsettings.json
└── Program.cs
```

İÇİNDEKİLER

1. Object Oriented Programming (OOP)
2. SOLID Prensipleri
3. C# Temel Yapılar
4. ASP.NET Core MVC Mimarisi
5. Proje Yapısı ve Yapılandırma

6. Routing
7. Razor View Engine
8. Controller'dan View'a Veri Taşıma
9. Tag Helpers
10. HTML ve Form Elemanları
11. Dependency Injection
12. Entity Framework Core
13. LINQ ile Veri Sorgulama
14. Model Validasyonu
15. TempData ve Bildirim Sistemi
16. Middleware ve Pipeline
17. DTO ve ViewModel
18. AJAX
19. Kategori CRUD Modülü
20. Hastane Randevu Modülü

1. Object Oriented Programming (OOP)

1.1 Genel Bakış

Nesne Yönelimli Programlama, yazılım problemlerini **nesne** adı verilen birimlere bölerek çözümlemeyi amaçlayan bir paradigmadır. Her nesne; veri (özellikler) ve bu veri üzerinde işlem yapabilen davranışları (metotları) bir arada barındırır.

OOP, büyük ölçekli yazılım projelerinde **modülerlik**, **yeniden kullanılabilirlik**, **bakım kolaylığı** ve **test edilebilirlik** sağlar. Karmaşık iş kurallarını gerçek dünya varlıklarına benzeyen sınıflara aktararak kodun anlaşılabilirliğini artırır.

C# tamamen nesne yönelimli bir dildir. Her tür `System.Object` 'ten türemiştir. Dilin temel taşları olan dört prensip Inheritance, Encapsulation, Abstraction ve Polymorphism'dir.

1.2 Dört Temel Prensip

Inheritance (Kalıtım)

Bir sınıfın başka bir sınıfın üyelerini devralmasıdır. Türetilen sınıfa **base class (temel sınıf)**, devralan sınıfa **derived class (türemiş sınıf)** denir.

```
// Models/Hayvan.cs
public class Hayvan
{
    public string Ad { get; set; }

    public void Nefes()
    {
        Console.WriteLine($"{Ad} nefes alıyor.");
    }
}

public class Kopek : Hayvan
{
    public void Havla()
    {
        Console.WriteLine($"{Ad}: Hav hav!");
    }
}

class Program
{
    static void Main()
    {
        var kopek = new Kopek { Ad = "Karabaş" };
        kopek.Nefes();
        kopek.Havla();
    }
}
```

Çıktı:

```
Karabaş nefes alıyor.
Karabaş: Hav hav!
```

Encapsulation (Kapsülleme)

Sınıf üyelerinin dış erişime karşı korunmasıdır. Erişim belirleyiciler (access modifiers) ile sağlanır.

Belirleyici	Erişim Kapsamı
<code>public</code>	Her yerden erişilebilir
<code>private</code>	Sadece tanımlandığı sınıf içinden
<code>internal</code>	Sadece aynı assembly (proje) içinden
<code>protected</code>	Tanımlandığı sınıf ve onu miras alan sınıflardan
<code>protected internal</code>	Aynı assembly veya miras alan tüm sınıflardan

Erişim belirleyici belirtilmediğinde sınıf üyeleri `private`, sınıfın kendisi `internal` kabul edilir.

```
// Models/BankaHesabi.cs
public class BankaHesabi
{
    private decimal _bakiye;    // Dışarıdan erişilemez

    public decimal Bakiye
    {
        get => _bakiye;
        set
        {
            if (value >= 0)
                _bakiye = value;
        }
    }

    public void ParaYatir(decimal miktar)
    {
        if (miktar > 0)
            _bakiye += miktar;
    }
}

class Program
{
    static void Main()
    {
        var hesap = new BankaHesabi();
        hesap.ParaYatir(500m);
        Console.WriteLine($"Bakiye: {hesap.Bakiye} TL");
    }
}
```

Çıktı:

Bakiye: 500 TL

Abstraction (Soyutlama)

Karmaşık iç detayları gizleyip yalnızca dışarıya açık olması gereken arayüzü sunmaktır. Kullanıcı "nasıl çalıştığını" değil, "ne yaptığını" bilir.

```
public abstract class Sekil
{
    public abstract double Alan();
}

public class Daire : Sekil
{
    public double Yaricap { get; set; }
    public override double Alan() => Math.PI * Yaricap * Yaricap;
}
```

Çıktı: (yok — yan etki: tip tanımı yapıldı, soyut metot türetilmiş sınıfta uygulandı.)

Soyutlama Interface ve Abstract Class ile sağlanır.

Polymorphism (Çok Biçimlilik)

Bir nesnenin birden fazla tipe gibi davranabilmesidir. `virtual` ile işaretlenen üst sınıf metodu, alt sınıfta `override` ile yeniden tanımlanır.

```
public class Sekil
{
    public virtual void Ciz()
    {
        Console.WriteLine("Şekil çiziliyor.");
    }
}

public class Daire : Sekil
{
    public override void Ciz()
    {
        Console.WriteLine("Daire çiziliyor.");
    }
}

public class Kare : Sekil
{
    public override void Ciz()
    {
        Console.WriteLine("Kare çiziliyor.");
    }
}

class Program
{
    static void Main()
    {
        Sekil[] sekiller = { new Daire(), new Kare(), new Sekil() };
        foreach (var s in sekiller)
            s.Ciz();
    }
}
```

Çıktı:

```
Daire çiziliyor.
Kare çiziliyor.
Şekil çiziliyor.
```

1.3 Bölüm Özeti

- OOP dört temel prensip üzerinde durur: Inheritance, Encapsulation, Abstraction, Polymorphism.
- Encapsulation erişim belirleyiciler aracılığıyla sağlanır.

- Abstraction Interface ve Abstract sınıflar ile uygulanır.
- Polymorphism `virtual` / `override` anahtar kelimeleriyle çalışır.
- C#'ta her tip `System.Object` 'ten türer; sınıflar tek miras (single inheritance) destekler ancak birden çok interface uygulayabilir.

2. SOLID Prensipleri

2.1 Genel Bakış

SOLID, nesne yönelimli yazılım tasarımını sürdürülebilir, esnek ve anlaşılır kılmak için tanımlanmış beş prensibin baş harflerinden oluşur. Robert C. Martin tarafından derlenmiştir. Bu prensipler bağımlılıkları azaltır, kodun bir alanını değiştirmenin diğerlerini bozmaması için disiplin sağlar.

2.2 Prensipler

Kısaltma	Türkçe Adı	Açıklama
SRP	Tek Sorumluluk	Her sınıfın değişmek için yalnızca tek bir nedeni olmalıdır.
OCP	Açık/Kapalı	Sınıflar genişlemeye açık, değişime kapalı olmalıdır.
LSP	Liskov Yerine Geçme	Alt sınıflar, üst sınıfların yerine geçebilmelidir.
ISP	Arayüz Ayırma	Büyük genel arayüzler yerine küçük ve spesifik arayüzler kullanılmalıdır.
DIP	Bağımlılık Tersine Çevirme	Yüksek seviye modüller, düşük seviye modüllere değil, soyutlamalara bağlı olmalıdır.

2.3 Single Responsibility Principle (SRP)

Bir sınıfın yalnızca **tek bir sorumluluğu** olmalıdır.

Anti-pattern:

```
// Tek sınıf üç farklı sorumluluk üstlenmiş
public class Ogrenci
{
    public void KayitOl() { }
    public void NotHesapla() { }
    public void EmailGonder() { }
}
```

İdiyomatik:

```

public class OgrenciKayitServisi
{
    public void KayitOl() { }
}

public class NotHesaplamaServisi
{
    public void Hesapla() { }
}

public class EmailServisi
{
    public void Gonder() { }
}

```

Çıktı: (yok — yan etki: sorumluluklar üç ayrı sınıfa dağıtıldı.)

2.4 Open / Closed Principle (OCP)

Sınıf davranışı **geniştirilebilmeli**, ancak kaynak kodu **değiştirilmemelidir**. Bu hedefe soyutlama (abstract/interface) ile ulaşılır.

```

public interface IIndirimStratejisi
{
    decimal Uygula(decimal tutar);
}

public class StandartIndirim : IIndirimStratejisi
{
    public decimal Uygula(decimal tutar) => tutar * 0.95m;
}

public class VipIndirim : IIndirimStratejisi
{
    public decimal Uygula(decimal tutar) => tutar * 0.80m;
}

public class Fatura
{
    private readonly IIndirimStratejisi _strateji;
    public Fatura(IIndirimStratejisi strateji) => _strateji = strateji;
    public decimal Hesapla(decimal tutar) => _strateji.Uygula(tutar);
}

```

Yeni bir indirim eklemek için `Fatura` sınıfı değiştirilmez; yeni bir `IIndirimStratejisi` uygulaması yazılır.

2.5 Liskov Substitution Principle (LSP)

Türetilmiş sınıflar, ana sınıfın yerine geçtiklerinde program davranışını bozmamalıdır. Yani temel sınıfı bekleyen herhangi bir kod, alt sınıf nesnesiyle de doğru çalışmalıdır.

2.6 Interface Segregation Principle (ISP)

Sınıflar kullanmadıkları metotları içeren arayüzleri uygulamak zorunda bırakılmamalıdır. Büyük tek arayüz yerine küçük ve odaklı arayüzler tercih edilir.

2.7 Dependency Inversion Principle (DIP)

Yüksek seviye modüller, düşük seviye modüllere değil, **soyutlamalara** bağımlı olmalıdır. ASP.NET Core'un Dependency Injection altyapısı bu prensibin pratik uygulamasıdır.

```
// Anti-pattern: doğrudan somut sınıfa bağımlılık
public class Rapor
{
    private PdfYazici _yazici = new PdfYazici();
}

// İdiyomatik: soyutlamaya bağımlılık
public interface IYazici
{
    void Yaz(string icerik);
}

public class Rapor
{
    private readonly IYazici _yazici;
    public Rapor(IYazici yazici) => _yazici = yazici;
}
```

2.8 Bölüm Özeti

- SOLID, OOP tasarımında bağımlılıkları azaltır, değişimi yerelleştirir.
- SRP her sınıfı tek amaçlı tutar; OCP genişletmeyi mevcut kodu bozmadan sağlar.
- LSP türetimi disipline eder; ISP arayüzleri küçük tutar.
- DIP, ASP.NET Core'un yerleşik DI mekanizmasıyla doğrudan ilgilidir (bkz. §11).

3. C# Temel Yapılar

3.1 Sınıf ve Nesne

Sınıf, nesne üretmek için kullanılan şablondur. Nesne, sınıftan üretilen somut örnektir.

```
public class Araba
{
    public string Marka { get; set; }
    public int Hiz { get; set; }

    public void Hizlan()
    {
        Hiz += 10;
        Console.WriteLine($"{Marka} - Hız: {Hiz}");
    }
}

class Program
{
    static void Main()
    {
        var araba = new Araba { Marka = "Toyota" };
        araba.Hizlan();
        araba.Hizlan();
    }
}
```

Çıktı:

```
Toyota - Hız: 10
Toyota - Hız: 20
```

3.2 Constructor

Sınıftan nesne üretilirken otomatik çalışan özel metottur. Bağımlılıkları zorunlu kılmak için kullanılır.

```

public class Hesap
{
    public int Bakiye { get; set; }

    public Hesap(int baslangic)
    {
        Bakiye = baslangic;
    }
}

class Program
{
    static void Main()
    {
        var h = new Hesap(100);    // parametre zorunlu
        Console.WriteLine($"Bakiye: {h.Bakiye}");
    }
}

```

Çıktı:

```
Bakiye: 100
```

Parametresiz `new Hesap()` çağırısı derleme zamanında hata verir:

```

error CS7036: There is no argument given that corresponds to the required parameter
'baslangic'

```

3.3 Property

Alanlara kontrollü erişim sağlar. Kısa formu otomatik property; uzun formu özel get/set mantığı için kullanılır.

```

public class Urun
{
    public int Id { get; set; }
    public string Ad { get; set; }
    public decimal Fiyat { get; set; }

    // Hesaplanmış property
    public string Ozet => $"{Ad} - {Fiyat} TL";
}

```

3.4 Interface

Sınıfların uyacağı sözleşmedir. Üye gövdesi içermez, yalnızca imza barındırır.

```
public interface IHesap
{
    void ParaYatir(int miktar);
    void ParaCek(int miktar);
}

public class BankaHesabi : IHesap
{
    public int Bakiye { get; set; }
    public void ParaYatir(int miktar) => Bakiye += miktar;
    public void ParaCek(int miktar) => Bakiye -= miktar;
}
```

Interface, kullanan kodun somut sınıfa değil sözleşmeye bağımlı kalmasını sağlar.

3.5 Tip Dönüşümü (Casting)

```
int sayi = 42;
string s = sayi.ToString();           // int → string
int parsed = int.Parse("123");        // string → int
int converted = Convert.ToInt32("7"); // string → int (null güvenli)
char c = (char)65;                    // int → char (ASCII)
Console.WriteLine($"{s} {parsed} {converted} {c}");
```

Çıktı:

```
42 123 7 A
```

3.6 Kontrol Yapıları

```
int gun = 3;
switch (gun)
{
    case 1: Console.WriteLine("Pazartesi"); break;
    case 2: Console.WriteLine("Salı"); break;
    case 3: Console.WriteLine("Çarşamba"); break;
    default: Console.WriteLine("Diğer"); break;
}

try
{
    int hatali = int.Parse("abc");
}
catch (FormatException ex)
{
    Console.WriteLine($"Hata: {ex.Message}");
}
```

Çıktı:

```
Çarşamba
Hata: The input string 'abc' was not in a correct format.
```

3.7 Döngüler

```
for (int i = 0; i < 3; i++)
    Console.WriteLine($"for: {i}");

string[] isimler = { "Ali", "Ayşe" };
foreach (var isim in isimler)
    Console.WriteLine($"foreach: {isim}");

int x = 0;
while (x < 2)
{
    Console.WriteLine($"while: {x}");
    x++;
}
```

Çıktı:

```

for: 0
for: 1
for: 2
foreach: Ali
foreach: Ayşe
while: 0
while: 1

```

3.8 Koleksiyon Arayüzleri: List, IEnumerable, ICollection

	List<T>	IEnumerable<T>	ICollection<T>
Türü	Sınıf	Interface	Interface
Veri okuma	Evet	Evet	Evet
Ekleme / silme	Evet	Hayır	Evet
Count	Evet	Hayır	Evet
İndeks erişimi	Evet	Hayır	Hayır

```

// Sadece okuma
IEnumerable<int> sayilar = new List<int> { 1, 2, 3 };

// Okuma + ekle/sil
ICollection<int> koleksiyon = new List<int>();
koleksiyon.Add(5);

// Tam koleksiyon
List<int> liste = new List<int> { 10, 20, 30 };
liste.Add(40);
int sayi = liste[0];
int adet = liste.Count;

```

Çıktı: (yok — yan etki: üç farklı koleksiyon tipi tanımlandı.)

3.9 Mini Sözlük

- **Sınıf (class):** Nesne üretmek için kullanılan şablon.
- **Nesne (object / instance):** Sınıftan üretilen somut örnek.
- **Property:** Alan değerini kontrollü olarak ifşa eden üye.
- **Constructor:** Nesne oluşturulurken otomatik çalışan metot.
- **Interface:** İmza topluluğu; uygulayan tipin uyacağı sözleşme.
- **Cast:** Bir tipi başka tipe dönüştürme işlemi.

4. ASP.NET Core MVC Mimarisi

4.1 Genel Bakış

MVC (Model-View-Controller), web uygulamalarında **Separation of Concerns (Sorumlulukların Ayrılması)** prensibine dayanan bir tasarım desendir. ASP.NET Core MVC, bu deseni .NET ekosisteminde uygulayan framework'tür. Uygulama; veri (Model), kullanıcı arayüzü (View) ve denetim mantığı (Controller) olmak üzere üç katmana ayrılır.

4.2 Katmanlar

Katman	Görev
Model	İş kuralları, veri yapısı ve veritabanı varlıkları. Diğer katmanlara bağımlı değildir; bağımsız test edilebilir.
View	Kullanıcıya gösterilen arayüz. HTML, CSS, JavaScript ve Razor sözdizimi içerir.
Controller	İsteği karşılayan, Model'den veri alan ve View'a aktaran beyin. HTTP request/response yaşam döngüsünün merkezindedir.

4.3 İstek Akışı

```
sequenceDiagram
    participant K as Kullanıcı
    participant R as Routing
    participant C as Controller
    participant M as Model / DB
    participant V as View

    K->>R: HTTP Request (URL)
    R->>C: Controller + Action belirle
    C->>M: Veri talebi
    M-->>C: Veri
    C->>V: return View(model)
    V-->>K: HTML Response
```

4.4 Bölüm Özeti

- MVC, sorumlulukları üç katmana böler.
- Model bağımsızdır; View kullanıcıya bakar; Controller iki tarafı birbirine bağlar.
- ASP.NET Core MVC, .NET üzerinde modüler ve test edilebilir web uygulamaları geliştirmenin standart yoludur.

5. Proje Yapısı ve Yapılandırma

5.1 Anahtar Dosyalar

Dosya / Klasör	Açıklama
<code>Program.cs</code>	Uygulamanın giriş noktası. Servis kaydı ve middleware yapılandırması burada yapılır.
<code>appsettings.json</code>	Bağlantı cümleleri, loglama ve diğer statik konfigürasyon.
<code>launchSettings.json</code>	Geliştirme ortamı: hangi profile, hangi portta çalışacağı.
<code>Controllers/</code>	Tüm controller sınıfları. Dosya adı <code>XxxController.cs</code> olmak zorunda.
<code>Models/</code>	Veri modelleri ve iş kuralları.
<code>Views/</code>	<code>.cshtml</code> Razor şablonları. Klasör adları controller adlarıyla eşleşir.
<code>Views/Shared/</code>	Layout ve tüm controller'larda kullanılabilecek partial view'lar.
<code>wwwroot/</code>	Statik dosyalar (CSS, JS, resimler). Dışarıdan doğrudan erişilebilir.
<code>Data/</code>	DbContext sınıfının tutulduğu klasör (geleneksel olarak).
<code>Migrations/</code>	EF Core tarafından oluşturulan migration dosyaları.

5.2 Program.cs Yapısı

```
// Program.cs
var builder = WebApplication.CreateBuilder(args);

// 1) Servis kayıtları (DI Container)
builder.Services.AddControllersWithViews()
    .AddRazorRuntimeCompilation(); // View değişiklikleri rebuild gerektirmesin

builder.Services.AddDbContext<ApplicationDbContext>(options =>
    options.UseSqlServer(
        builder.Configuration.GetConnectionString("DefaultConnection")
    )
);

var app = builder.Build();

// 2) Middleware (pipeline)
if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Home/Error");
    app.UseHsts();
}

app.UseHttpsRedirection();
app.UseStaticFiles();
app.UseRouting();
app.UseAuthorization();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");

app.Run();
```

Çıktı: (yok — yan etki: uygulama Kestrel sunucusunda başlar, gelen istekler tanımlanan pipeline üzerinden işlenir.)

5.3 appsettings.json

```
{
  "ConnectionStrings": {
    "DefaultConnection": "YOUR_CONNECTION_STRING"
  },
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*"
}
```

`GetConnectionString("DefaultConnection")` çağrısı, **ConnectionStrings** bloğu içindeki anahtara göre çalışır. Bloğun adı sabittir; özel bir anahtar adı kullanmak istenirse `Configuration["..."]` ile okumak gerekir.

5.4 SDK ve Runtime Farkı

Amaç	Geliştirme araçları (compiler, CLI, kütüphaneler)	Uygulamayı çalıştırmak için gereken minimum sistem
Kurulu olmalı	Geliştirici makinesi	Sunucu / son kullanıcı makinesi

5.5 Bölüm Özeti

- `Program.cs` iki bölümden oluşur: servis kaydı ve pipeline yapılandırması.
- Hassas konfigürasyonlar `appsettings.json` içine taşınır, kod içine yazılmaz.
- `ConnectionStrings` anahtarı çerçeve tarafından özel olarak tanınır.
- ASP.NET Core uygulamaları aslında birer konsol uygulamasıdır; Kestrel sunucusunu gömülü olarak barındırırlar.

6. Routing

6.1 Genel Bakış

Routing, gelen URL'i hangi controller ve action metoduyla eşleştireceğine karar veren mekanizmadır. ASP.NET Core MVC'de `MapControllerRoute` ile yapılandırılır.

6.2 Sözdizimi

```
// Program.cs
app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");
```

- `controller=Home` → Controller verilmemişse `HomeController` kullanılır.
- `action=Index` → Action verilmemişse `Index()` çalışır.
- `id?` → Soru işareti `id` parametresinin opsiyonel olduğunu belirtir.

6.3 URL → Controller Eşleştirme Örnekleri

URL	Controller	Action	id
/	Home	Index	—
/Category	Category	Index	—
/Category/Create	Category	Create	—
/Category/Edit/5	Category	Edit	5

6.4 Action Method Sözdizimi

```
// Controllers/CategoryController.cs
public class CategoryController : Controller
{
    public IActionResult Index()           // /Category
    {
        return View();
    }

    public IActionResult Edit(int? id)      // /Category/Edit/3
    {
        return View();
    }
}
```

6.5 HTTP Yöntem Attribute'ları

```
public IActionResult Create()           // GET (varsayılan)
{
    return View();
}

[HttpPost]
[ValidateAntiForgeryToken]
public IActionResult Create(Category obj) // POST
{
    return RedirectToAction("Index");
}
```

`[ValidateAntiForgeryToken]`, Cross-Site Request Forgery (CSRF) saldırılarını engellemek için form gönderimlerinde otomatik üretilen gizli anahtarları doğrular.

6.6 Action Result Tipleri

Dönüş	Açıklama
<code>View()</code>	İlgili <code>.cshtml</code> sayfasını döndürür.
<code>View("AdView", model)</code>	Belirli bir view'ı modelle birlikte döndürür.
<code>RedirectToAction("Index")</code>	Başka bir action'a 302 yönlendirme yapar.
<code>RedirectToAction("Index", "Home")</code>	Farklı controller'daki action'a yönlendirme.
<code>Json(data)</code>	JSON formatında veri döndürür.
<code>Content("...")</code>	Düz metin döner.
<code>NotFound()</code>	HTTP 404 döner.
<code>BadRequest()</code>	HTTP 400 döner.

6.7 Bölüm Özeti

- Routing pattern'i `controller/action/id?` formatındadır.
- Controller adı ve action adı zorunlu; id opsiyoneldir.
- `IActionResult`, View / Redirect / Json gibi farklı dönüş türlerini soyutlayan ortak arayüzdür.
- POST eylemleri için `[HttpPost]` ve `[ValidateAntiForgeryToken]` birlikte kullanılır.

7. Razor View Engine

7.1 Genel Bakış

Razor, C# kodunu HTML içine gömmeye imkanı sağlayan bir şablon motorudur. Tarayıcı C# anlamadığı için Razor, sunucu tarafında bu kodları HTML'e çevirir.

7.2 Temel Sözdizimi

```
@{
    var baslik = "Ürün Listesi";
    var urunler = new[] { "Laptop", "Mouse", "Klavye" };
}

<h1>@baslik</h1>

<ul>
    @foreach (var u in urunler)
    {
        <li>@u</li>
    }
</ul>

@if (urunler.Length == 0)
{
    <p>Ürün yok.</p>
}
```

Üretilen HTML:

```
<h1>Ürün Listesi</h1>
<ul>
    <li>Laptop</li>
    <li>Mouse</li>
    <li>Klavye</li>
</ul>
```

7.3 Model Tanımlama

View'ın en üstüne `@model` direktifi konur. Direktif **küçük harfle** yazılır; veri kullanırken `Model` (büyük M) ifadesi kullanılır.

```
@model List<Category>

@foreach (var item in Model)
{
    <p>@item.Name - @item.DisplayOrder</p>
}
```

7.4 Layout

Tüm sayfalarda ortak olan iskelet `Views/Shared/_Layout.cshtml` dosyasında bulunur. `@RenderBody()` ifadesi, alt sayfanın içeriğinin basılacağı yeri belirler.

```
<!-- Views/Shared/_Layout.cshtml -->
<!DOCTYPE html>
<html>
<head>
    <link rel="stylesheet" href="~/css/bootstrap-theme.css" />
    @await RenderSectionAsync("Css", required: false)
</head>
<body>
    <header> ... </header>

    <div class="container">
        @RenderBody()
    </div>

    <footer> ... </footer>

    <script src="~/lib/jquery/dist/jquery.min.js"></script>
    @await RenderSectionAsync("Scripts", required: false)
</body>
</html>
```

7.5 _ViewStart.cshtml

Tüm sayfalar için varsayılan layout'u belirler.

```
@{
    Layout = "_Layout";
}
```

7.6 _ViewImports.cshtml

Tüm view'larda otomatik geçerli `using` ifadeleri ve tag helper aktivasyonu için kullanılır.

```
@using BulkyBookWeb
@using BulkyBookWeb.Models
@addTagHelper *, Microsoft.AspNetCore.Mvc.TagHelpers
```

7.7 Sections

Belirli bir alana sayfa bazlı içerik enjekte etmek için `@section` kullanılır.

```
<!-- Views/Category/Index.cshtml -->
@section Scripts {
    <script>
        console.log("Index sayfası yüklendi.");
    </script>
}
```

Layout tarafı:

```
@await RenderSectionAsync("Scripts", required: false)
```

`required: false` ile bu section opsiyonel olur.

7.8 Partial View

Sayfanın tekrar eden parçaları için kullanılır. Genellikle `Views/Shared/` altına `_AdView.cshtml` adıyla konur.

```
<!-- Views/Shared/_Notification.cshtml -->
@if (TempData["success"] != null)
{
    <script>toastr.success('@TempData["success"]');</script>
}
```

Kullanım:

```
<partial name="_Notification" />
```

Alternatif:

```
@await Html.PartialAsync("_Notification")
```

7.9 Bölüm Özeti

- Razor, `@` karakteri ile C# kodunu HTML içine taşır.
- Direktif `@model`, kullanım `Model` 'dir; büyük-küçük harf farkı önemlidir.
- Layout sayfa iskeletini; ViewStart varsayılan layout'u; ViewImports global using'leri tanımlar.

- Partial View'lar tekrar eden parçaları tek bir yerden yönetmeyi sağlar.
- Section'lar layout'a sayfa bazlı enjeksiyon imkanı verir.

8. Controller'dan View'a Veri Taşıma

8.1 Genel Bakış

Controller'da hazırlanan verinin View'a aktarılması için dört temel yöntem kullanılır. En profesyonel ve hata payı en düşük olan **Model / ViewModel** yaklaşımıdır.

8.2 ViewBag

Dinamik yapıda, hızlı veri taşıma için.

```
// Controllers/HomeController.cs
public IActionResult Index()
{
    ViewBag.Mesaj = "Hoş geldiniz!";
    return View();
}
```

```
<!-- Views/Home/Index.cshtml -->
<h1>@ViewBag.Mesaj</h1>
```

8.3 ViewData

Dictionary tabanlı.

```
ViewData["Mesaj"] = "Selam";
```

```
<h1>@ViewData["Mesaj"]</h1>
```

8.4 TempData

Bir sonraki istek için saklanır. Genellikle yönlendirmelerden sonra bildirim mesajı göstermek için kullanılır. Tek seferlik okuma yapılır; sayfa yenilenince kaybolur.

```
TempData["success"] = "Kayıt başarılı!";
return RedirectToAction("Index");
```

8.5 Model / ViewModel

En önerilen yöntem. Strongly typed olduğu için derleme zamanı güvenliği sağlar.

```
public IActionResult Index()
{
    IEnumerable<Category> categories = _db.Categories;
    return View(categories);
}
```

```
@model IEnumerable<Category>

@foreach (var c in Model)
{
    <p>@c.Name</p>
}
```

8.6 Karşılaştırma

Yöntem	Yapı	Güvenli mi?	Redirect sonrası yaşar mı?
ViewBag	Dinamik	Hayır	Hayır
ViewData	Dictionary	Hayır	Hayır
TempData	Dictionary	Hayır	Evet (1 request)
Model	Strongly typed	Evet	—

8.7 Bölüm Özeti

- ViewBag ve ViewData, aynı istek içinde küçük veri taşımak için uygundur.
- TempData, redirect sonrası mesaj göstermek için tasarlanmıştır.
- Asıl iş verisi her zaman Model veya ViewModel ile taşınır.

9. Tag Helpers

9.1 Genel Bakış

Tag Helper'lar, HTML etiketlerine `asp-` öneki ile sunucu tarafı yetenekleri ekleyen yapılardır. HTML görünümünde yazıldıkları için okunması kolaydır ve Razor'ın eski HTML Helper sözdiziminin yerini almıştır.

Aktivasyon `_ViewImports.cshtml` üzerinden yapılır:

```
@addTagHelper *, Microsoft.AspNetCore.Mvc.TagHelpers
```

9.2 Yaygın Tag Helper'lar

Tag Helper	Kullanım	Görev
<code>asp-controller</code>	<code><a asp-controller="Category" ...></code>	Hangi controller'a gidileceği
<code>asp-action</code>	<code><a asp-action="Edit" ...></code>	Hangi action'a gidileceği
<code>asp-route-id</code>	<code><a asp-route-id="@obj.Id" ...></code>	URL'e route parametresi ekler
<code>asp-for</code>	<code><input asp-for="Name" /></code>	Input/label'ı model property'sine bağlar
<code>asp-items</code>	<code><select asp-items="ViewBag.List"></code>	Dropdown öğelerini doldurur
<code>asp-validation-for</code>	<code></code>	Property için hata mesajını gösterir
<code>asp-validation-summary</code>	<code><div asp-validation-summary="All"></code>	Tüm hataları üstte özetler

9.3 Bağlantı (Link)

```
<a asp-controller="Category" asp-action="Edit"
  asp-route-id="@obj.Id" class="btn btn-primary">
  Düzenle
</a>
```

Üretilen HTML:

```
<a href="/Category/Edit/3" class="btn btn-primary">Düzenle</a>
```

`asp-route-id` ile URL'e parametre eklenir. Controller şu şekilde alır:

```
public IActionResult Edit(int? id)
{
    // URL'den gelen id buraya gelir
}
```

9.4 Form

```
@model Category
```

```
<form method="post" asp-action="Create">

    <div class="mb-3">
        <label asp-for="Name"></label>
        <input asp-for="Name" class="form-control" />
        <span asp-validation-for="Name" class="text-danger"></span>
    </div>

    <div class="mb-3">
        <label asp-for="DisplayOrder"></label>
        <input asp-for="DisplayOrder" class="form-control" />
        <span asp-validation-for="DisplayOrder" class="text-danger"></span>
    </div>

    <button type="submit" class="btn btn-primary">Oluştur</button>
    <a asp-action="Index" class="btn btn-secondary">Listeye Dön</a>
</form>
```

`asp-for="Name"` yazıldığında: - Label otomatik olarak property adını basar (varsa `[DisplayName]` değerini). - Input'ta `id`, `name` ve gerekirse `type` attribute'ları otomatik atanır. - Form gönderildiğinde input değeri ilgili property'ye bağlanır.

9.5 Hidden Input

Kullanıcı görmediği ancak POST sırasında gönderilmesi gereken alanlar için.

```
<input type="hidden" asp-for="Id" />
```

Edit ve Delete senaryolarında `Id` değerini taşımak için kullanılır.

9.6 Select (Dropdown)

```
ViewBag.BransListesi = new SelectList(_db.Branslar, "Id", "BransAdi");
return View();
```

```
<select asp-for="BransId" asp-items="ViewBag.BransListesi" class="form-control">
    <option value="">-- Branş Seçiniz --</option>
</select>
```

9.7 Validation Summary

```
<div asp-validation-summary="All" class="text-danger"></div>
```

Değer	Açıklama
All	Tüm hataları gösterir (model + property seviyesi)
ModelOnly	Sadece model seviyesindeki (özel custom) hatalar
None	Hiçbir özet gösterilmez

9.8 Bölüm Özeti

- Tag Helper'lar `asp-` önekiyle gelir ve sunucu tarafında işlenir.
- `asp-for` modele bağlama, `asp-controller` / `asp-action` / `asp-route-id` URL üretimi için kullanılır.
- Validation Tag Helper'ları (`validation-for`, `validation-summary`) form doğrulamayı otomatikleştirir.

10. HTML ve Form Elemanları

10.1 Genel Bakış

ASP.NET Core MVC'nin View tarafı sonuç olarak HTML üretir. Razor ve Tag Helper'ların hangi HTML'i ürettiğini anlamak için temel HTML yapısının bilinmesi gerekir.

10.2 Temel Sayfa İskelet

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8" />
  <title>Sayfa Başlığı</title>
</head>
<body>
  <h1>Merhaba</h1>
</body>
</html>
```

10.3 Sık Kullanılan Etiketler

Etiket	Görev
<code><h1></code> – <code><h6></code>	Başlıklar
<code><p></code>	Paragraf
<code></code>	Bağlantı
<code></code>	Resim
<code></code> / <code></code> + <code></code>	Sırasız / sıralı liste
<code><table></code> , <code><tr></code> , <code><th></code> , <code><td></code>	Tablo yapısı
<code><div></code>	Blok kapsayıcı
<code></code>	Satır içi kapsayıcı

10.4 Form Elemanları

```

<form action="/Category/Create" method="post">

    <label for="Name">Ad:</label>
    <input type="text" id="Name" name="Name" />

    <input type="email" name="Email" />
    <input type="number" name="Yas" min="1" max="100" />
    <input type="password" name="Sifre" />
    <input type="checkbox" name="Onay" />
    <input type="radio" name="Cinsiyet" value="E" />
    <input type="hidden" name="Id" value="5" />

    <textarea name="Aciklama" rows="4"></textarea>

    <select name="Sehir">
        <option value="1">İstanbul</option>
        <option value="2">Ankara</option>
    </select>

    <button type="submit">Kaydet</button>
    <button type="reset">Temizle</button>

</form>

```

Üretilen istek: `POST /Category/Create` ve `name` attribute'ları form data anahtarı olarak gönderilir.

10.5 Disabled vs Readonly

Attribute	Görünüm	POST'a dahil mi?
<code>disabled</code>	Soluk, tıklanamaz	Hayır
<code>readonly</code>	Normal görünür, değiştirilemez	Evet

Delete onay sayfalarında alanları `disabled` yaparak kullanıcının değiştiremeyeceği bir özet gösterimi sağlanır.

10.6 Form Submit'te ASP.NET MVC Bağlama (Model Binding)

```
<input type="text" name="Name" value="Laptop" />
<input type="number" name="DisplayOrder" value="3" />
```

```
[HttpPost]
public IActionResult Create(Category obj)
{
    // obj.Name = "Laptop"
    // obj.DisplayOrder = 3
}
```

Form input'undaki `name` attribute'u, model property adıyla eşleşmek zorundadır. Tag Helper `asp-for="Name"` bu eşleşmeyi otomatik kurar.

10.7 Bölüm Özeti

- Razor ve Tag Helper'ların ürettiği çıktı sonuçta standart HTML'dir.
- Form input'larının `name` attribute'u, Controller tarafındaki model property adıyla eşleşmelidir.
- `disabled` ve `readonly` farkı: birinci POST'a dahil edilmez, ikincisi edilir.

11. Dependency Injection

11.1 Genel Bakış

Dependency Injection (DI), bir nesnenin ihtiyaç duyduğu bağımlılıkları kendi içinde oluşturmak yerine **dışarıdan** almasıdır. ASP.NET Core, DI'ı çekirdek bir özellik olarak yerleşik biçimde sunar; bu, SOLID'in Dependency Inversion prensibinin pratik uygulamasıdır.

11.2 Bağımlılık Sorunu

Aşağıdaki örnekte sınıf, somut bir sınıfa doğrudan bağlıdır:

```
public class UrunController : Controller
{
    private OrderService _service;

    public UrunController()
    {
        _service = new OrderService(); // somut sınıfa bağımlılık
    }
}
```

`OrderService` değişirse, `UrunController` da değiştirilmek zorundadır. Birim test yazmak güçleşir; çünkü gerçek `OrderService` her zaman çalışacaktır.

11.3 Constructor Injection ile Çözüm

```
public class UrunController : Controller
{
    private readonly IUrunService _service;

    public UrunController(IUrunService service)
    {
        _service = service;
    }

    public IActionResult Index()
    {
        var urunler = _service.GetAll();
        return View(urunler);
    }
}
```

Burada controller, arayüze (`IUrunService`) bağımlıdır. Hangi somut sınıfın geleceğini DI Container belirler.

11.4 Servis Kaydı

```
// Program.cs
builder.Services.AddScoped<IUrunService, UrunService>();
```

`IUrunService` istendiğinde framework, `UrunService` örneği üretir ve constructor'a enjekte eder.

11.5 Yaşam Süreleri (Lifetimes)

Metot	Yaşam Süresi
<code>AddTransient<TService, TImpl>()</code>	Her istendiğinde yeni örnek. Hafif, durumsuz servisler için.
<code>AddScoped<TService, TImpl>()</code>	Her HTTP isteği için tek örnek. EF Core DbContext bu kapsamda kaydedilir.
<code>AddSingleton<TService, TImpl>()</code>	Uygulama ömrü boyunca tek örnek. Cache, konfigürasyon vb. için.

11.6 Çoklu Servis Enjeksiyonu

Bir controller birden fazla bağımlılığı constructor üzerinden alabilir:

```
public class SiparisController : Controller
{
    private readonly IUrunService _urunService;
    private readonly ISiparisService _siparisService;

    public SiparisController(IUrunService urunService, ISiparisService siparisService)
    {
        _urunService = urunService;
        _siparisService = siparisService;
    }
}
```

11.7 Servis İçinde Servis Enjeksiyonu

Servisler birbirini de enjekte edebilir:

```
public class SiparisService : ISiparisService
{
    private readonly IUrunService _urunService;

    public SiparisService(IUrunService urunService)
    {
        _urunService = urunService;
    }
}
```

DI Container, bağımlılık zincirini otomatik çözer. `Program.cs` içinde ilgili tüm tipler kayıtlı olduğu sürece manuel `new` çağrısı gerekmez.

11.8 Akış Şeması

```
sequenceDiagram
    participant App as ASP.NET Core
    participant DI as DI Container
    participant C as UrunController
    participant S as UrunService

    App->>DI: UrunController oluştur
    DI->>DI: Constructor'da IUrunService ister
    DI->>S: UrunService örneği üret
    DI->>C: Constructor'a inject et
    C-->>App: Hazır controller
```

11.9 Bölüm Özeti

- DI, bağımlılıkları soyutlamalara taşıyarak değişimi yerelleştirir ve testi kolaylaştırır.
- Constructor injection en yaygın yöntemdir; `readonly` field garantisi sağlar.
- Lifetime seçimi servis türüne göre yapılır (DbContext → Scoped; durumsuz yardımcı servis → Transient; cache → Singleton).
- Servisler arası bağımlılıklar da DI Container tarafından çözülür.

12. Entity Framework Core

12.1 Genel Bakış

Entity Framework Core, .NET için açık kaynaklı ve cross-platform bir ORM'dir (Object-Relational Mapper). SQL yazmadan C# nesneleri üzerinden veritabanı işlemleri yapmayı sağlar. Code First yaklaşımında modeller önce C# sınıfı olarak yazılır, ardından migration ile veritabanı oluşturulur.

12.2 NuGet Paketleri

Paket	Görev
Microsoft.EntityFrameworkCore	Çekirdek ORM özellikleri
Microsoft.EntityFrameworkCore.SqlServer	SQL Server sağlayıcısı
Microsoft.EntityFrameworkCore.Tools	Add-Migration , Update-Database komutları

12.3 Model (Entity)

Her sınıf bir tabloyu temsil eder. Data Annotation attribute'ları ile kolon kuralları tanımlanır.

```
// Models/Category.cs
using System.ComponentModel;
using System.ComponentModel.DataAnnotations;

public class Category
{
    [Key]
    public int Id { get; set; }

    [Required]
    [MaxLength(100)]
    public string Name { get; set; }

    [DisplayName("Display Order")]
    [Range(1, 100, ErrorMessage = "Display Order must be between 1 and 100.")]
    public int DisplayOrder { get; set; }

    public DateTime CreatedDateTime { get; set; } = DateTime.Now;
}
```

12.4 DbContext

DbContext, veritabanı oturumunu temsil eden sınıftır. Her `DbSet<T>` bir tabloya karşılık gelir.

```
// Data/ApplicationDbContext.cs
using Microsoft.EntityFrameworkCore;

public class ApplicationDbContext : DbContext
{
    public ApplicationDbContext(DbContextOptions<ApplicationDbContext> options)
        : base(options) { }

    public DbSet<Category> Categories { get; set; }
}
```

12.5 DbContext Kaydı

```
// Program.cs
builder.Services.AddDbContext<ApplicationDbContext>(options =>
    options.UseSqlServer(
        builder.Configuration.GetConnectionString("DefaultConnection")
    )
);
```

12.6 Migration

Package Manager Console'da:

```
Add-Migration "AddCategoryToDatabase"
Update-Database
```

`Add-Migration` komutu, `Migrations/` klasöründe içinde `Up()` ve `Down()` metotları bulunan bir dosya üretir.

```
// Migrations/<timestamp>_AddCategoryToDatabase.cs
public partial class AddCategoryToDatabase : Migration
{
    protected override void Up(MigrationBuilder migrationBuilder)
    {
        migrationBuilder.CreateTable(
            name: "Categories",
            columns: table => new
            {
                Id = table.Column<int>(nullable: false)
                    .Annotation("SqlServer:Identity", "1, 1"),
                Name = table.Column<string>(maxLength: 100, nullable: false),
                DisplayOrder = table.Column<int>(nullable: false),
                CreatedDateTime = table.Column<DateTime>(nullable: false)
            },
            constraints: table =>
            {
                table.PrimaryKey("PK_Categories", x => x.Id);
            });
    }

    protected override void Down(MigrationBuilder migrationBuilder)
    {
        migrationBuilder.DropTable(name: "Categories");
    }
}
```

`Update-Database` komutu bu migration'ı veritabanına uygular ve `__EFMigrationsHistory` tablosunda işaretler. Yeniden çalıştırıldığında daha önce uygulanmış migration'lar atlanır.

12.7 Sürüm Notu

Sürüm notu: EF Framework (klasik) ile EF Core farklı kütüphanelerdir. EF Core (.NET Core / .NET 5+) cross-platform ve aktif geliştirmededir. Bu belge EF Core esas alınarak yazılmıştır.

12.8 Yaklaşımlar

Yaklaşım	Akış
Code First	C# modellerini yaz → Migration → Veritabanı oluşur
Database First	Var olan veritabanı → Scaffold-DbContext → C# modelleri üretilir

Hastane projesinde Database First yaklaşımı kullanılmıştır:

```
Scaffold-DbContext "YOUR_CONNECTION_STRING"
  Microsoft.EntityFrameworkCore.SqlServer
  -OutputDir Models
```

12.9 Data Annotations

Attribute	Görev
[Key]	Primary key
[Required]	NOT NULL
[MaxLength(n)]	Maksimum karakter sayısı
[StringLength(n, MinimumLength=m)]	Min ve maks uzunluk
[Range(min, max)]	Sayısal aralık
[EmailAddress]	Email formatı
[DisplayName("Ad")]	View'da gösterilecek etiket

12.10 Bölüm Özeti

- EF Core, SQL yazmadan veritabanı işlemleri yapmaya olanak tanıyan bir ORM'dir.
- DbContext bir oturumu temsil eder; DbSet'ler tabloları temsil eder.
- Code First: Model → Migration → DB. Database First: DB → Scaffold → Model.
- Data Annotations hem veritabanı şemasını hem form validasyonunu etkiler.

13. LINQ ile Veri Sorgulama

13.1 Genel Bakış

LINQ (Language Integrated Query), C# içinde koleksiyonlar ve veritabanı üzerinde standart sorgu yazmayı sağlar. EF Core, LINQ ifadelerini SQL'e dönüştürür.

13.2 Temel Sorgular

```
// READ - hepsini getir
var hepsi = _db.Categories.ToList();

// READ - tek kayıt (primary key ile)
var tek = _db.Categories.Find(id);

// Where ile filtreleme
var filtreli = _db.Categories
    .Where(c => c.DisplayOrder > 5)
    .ToList();

// OrderBy ile sıralama
var sirali = _db.Categories
    .OrderBy(c => c.Name)
    .ToList();

// Select ile projection
var sadeIsim = _db.Categories
    .Select(c => c.Name)
    .ToList();
```

13.3 Tek Kayıt Getirme Metotları

Metot	Birden çok kayıt varsa	Kayıt bulunamazsa
<code>Find(id)</code>	(primary key ile çağrı)	<code>null</code>
<code>FirstOrDefault(predicate)</code>	İlkini döner	<code>null</code>
<code>SingleOrDefault(predicate)</code>	Exception fırlatır	<code>null</code>
<code>First(predicate)</code>	İlkini döner	Exception
<code>Single(predicate)</code>	Exception	Exception

```
var c1 = _db.Categories.Find(id);
var c2 = _db.Categories.FirstOrDefault(c => c.Id == id);
var c3 = _db.Categories.SingleOrDefault(c => c.Id == id);
```

ID primary key olduğu için pratikte `Find(id)` en kısa yoldur.

13.4 CRUD İşlemleri

```
// CREATE
_db.Categories.Add(category);
_db.SaveChanges();

// UPDATE
_db.Categories.Update(category); // Id'ye bakar
_db.SaveChanges();

// DELETE
_db.Categories.Remove(category);
_db.SaveChanges();

// AddRange
_db.Categories.AddRange(listeKategori);
```

`SaveChanges()` çağrılmazsa hiçbir değişiklik veritabanına yansımaz.

13.5 İki Tabloyu Birleştirme (Join)

İki yöntem vardır: Navigation Property üzerinden `Include` veya açık LINQ `join`.

Yöntem 1: Include (Eager Loading)

```
var doktorlar = _context.Doktorlar
    .Include(d => d.Brans)
    .ToList();

foreach (var d in doktorlar)
{
    Console.WriteLine($"{d.AdSoyad} - {d.Brans.BransAdi}");
}
```

Üretilen SQL:

```
SELECT d.*, b.*
FROM Doktorlar d
LEFT JOIN Branslar b ON d.BransId = b.Id
```

Birden fazla seviye için `ThenInclude`:

```
var randevular = _context.Randevular
    .Include(r => r.Hasta)
    .Include(r => r.Doktor)
    .ThenInclude(d => d.Brans)
    .ToList();
```

Yöntem 2: LINQ Join (Query Syntax)

```
var sonuc = from d in _context.Doktorlar
            join b in _context.Branslar
            on d.BransId equals b.Id
            select new
            {
                DoktorAd = d.AdSoyad,
                BransAdi = b.BransAdi
            };
```

Yöntem 3: LINQ Join (Method Syntax)

```
var sonuc = _context.Doktorlar
    .Join(
        _context.Branslar,
        d => d.BransId,
        b => b.Id,
        (d, b) => new { DoktorAd = d.AdSoyad, BransAdi = b.BransAdi }
    )
    .ToList();
```

13.6 Include vs Join Farkı

	Include	Join
Tam nesne döner mi?	Evet (Doktor ve içindeki Brans)	Hayır (sadece seçilen alanlar)
Navigation Property gerekli mi?	Evet	Hayır
Anonim tip dönüşü kolay mı?	Hayır	Evet (Select new {...})

13.7 Bölüm Özeti

- LINQ, C# içinde tek bir sorgu dili sağlar; EF Core bunu SQL'e çevirir.
- `Find` primary key için en kısa yol; `FirstOrDefault` esnek filtreleme.
- CRUD işlemleri `Add`, `Update`, `Remove` ile yapılır; her durumda `SaveChanges()` çağrılır.
- İlişkili veri çekme için Navigation Property varsa `Include`, yoksa LINQ `join` tercih edilir.

14. Model Validasyonu

14.1 Genel Bakış

ASP.NET Core MVC, model validasyonunu hem sunucu tarafında hem de istemci tarafında otomatikleştir. Validation kuralları Data Annotation attribute'ları ile tanımlanır.

14.2 Server-Side Validasyon

Controller, gelen modelin geçerliliğini `ModelState.IsValid` ile kontrol eder.

```
[HttpPost]
[ValidateAntiForgeryToken]
public IActionResult Create(Category obj)
{
    if (ModelState.IsValid)
    {
        _db.Categories.Add(obj);
        _db.SaveChanges();
        TempData["success"] = "Category created successfully";
        return RedirectToAction("Index");
    }
    return View(obj);
}
```

Model geçersizse `obj` yeniden View'a gönderilir; kullanıcı doldurduğu alanları kaybetmez.

14.3 Client-Side Validasyon

`<span asp-validation-for="...">` etiketi, ilgili property için JavaScript tabanlı doğrulama mesajını gösterir. Validation script'leri sayfaya eklenmelidir.

```
@model Category
```

```
<form method="post" asp-action="Create">
    <div class="mb-3">
        <label asp-for="Name"></label>
        <input asp-for="Name" class="form-control" />
        <span asp-validation-for="Name" class="text-danger"></span>
    </div>

    <div class="mb-3">
        <label asp-for="DisplayOrder"></label>
        <input asp-for="DisplayOrder" class="form-control" />
        <span asp-validation-for="DisplayOrder" class="text-danger"></span>
    </div>

    <button type="submit" class="btn btn-primary">Create</button>
</form>

@section Scripts {
    <partial name="_ValidationScriptsPartial" />
}
```

14.4 Validation Summary

Tüm hataları üstte özet halinde göstermek için:

```
<div asp-validation-summary="All" class="text-danger"></div>
```

Mod	Görünen Hatalar
All	Property + model seviyesi
ModelOnly	Sadece model seviyesi (özel hatalar)
None	Hiçbiri

14.5 Özel (Custom) Hata Ekleme

Modeli incelemekten veya farklı bir iş kuralı uygulamak için Controller içinde manuel hata eklenebilir:

```
[HttpPost]
public IActionResult Create(Category obj)
{
    if (obj.Name == obj.DisplayOrder.ToString())
    {
        // Validation Summary'de görünür (key herhangi bir property değil)
        ModelState.AddModelError("customError",
            "The DisplayOrder cannot exactly match the Name.");

        // Veya Name property'sinin altında görünür
        ModelState.AddModelError("Name",
            "The DisplayOrder cannot exactly match the Name.");
    }

    if (ModelState.IsValid)
    {
        _db.Categories.Add(obj);
        _db.SaveChanges();
        return RedirectToAction("Index");
    }
    return View(obj);
}
```

Aynı anahtar (key) iki kez kullanılmamalıdır.

14.6 Data Annotations Örnekleri

```
public class Category
{
    [Key]
    public int Id { get; set; }

    [Required(ErrorMessage = "Name is required.")]
    [MaxLength(30)]
    public string Name { get; set; }

    [DisplayName("Display Order")]
    [Range(1, 100, ErrorMessage = "Display Order must be between 1 and 100.")]
    public int DisplayOrder { get; set; }
}
```

`[DisplayName]` ile property adı değiştirilince hem label hem hata mesajındaki etiket otomatik güncellenir.

14.7 Bölüm Özeti

- Validasyon hem sunucu (`ModelState.IsValid`) hem istemci (Tag Helper + script) tarafında çalışır.
- Data Annotation'lar tek noktada tanımlanır ve View, Controller, EF Core tarafından paylaşılır.
- `ModelState.AddModelError` özel iş kuralları için manuel hata eklemenin yoludur.
- Form gönderim sonrası model geçersizse `return View(obj)` ile kullanıcının verisi korunur.

15. TempData ve Bildirim Sistemi

15.1 Genel Bakış

TempData, bir HTTP isteğinden bir sonrakine veri taşımak için tasarlanmıştır. Redirect sonrası kullanıcıya başarı veya hata mesajı göstermek için idealdir. Veri okunduktan veya bir sonraki istek tamamlandıktan sonra otomatik silinir.

15.2 Controller'da TempData Kullanımı

```
[HttpPost]
public IActionResult Create(Category obj)
{
    if (ModelState.IsValid)
    {
        _db.Categories.Add(obj);
        _db.SaveChanges();
        TempData["success"] = "Category created successfully";
        return RedirectToAction("Index");
    }
    return View(obj);
}

[HttpPost]
public IActionResult Edit(Category obj)
{
    if (ModelState.IsValid)
    {
        _db.Categories.Update(obj);
        _db.SaveChanges();
        TempData["success"] = "Category updated successfully";
        return RedirectToAction("Index");
    }
    return View(obj);
}

[HttpPost, ActionName("Delete")]
public IActionResult DeletePOST(int? id)
{
    var obj = _db.Categories.Find(id);
    if (obj == null) return NotFound();

    _db.Categories.Remove(obj);
    _db.SaveChanges();
    TempData["success"] = "Category deleted successfully";
    return RedirectToAction("Index");
}
```

15.3 Toastr.js Entegrasyonu

CSS ve JS Bağlama

```
<link rel="stylesheet"
      href="https://cdnjs.cloudflare.com/ajax/libs/toastr.js/latest/toastr.min.css" />

<script src="~/lib/jquery/dist/jquery.min.js"></script>
<script src="https://cdnjs.cloudflare.com/ajax/libs/toastr.js/latest/toastr.min.js"></scrip
t>
```

Bildirim Partial View

```
<!-- Views/Shared/_Notification.cshtml -->
@if (TempData["success"] != null)
{
    <script type="text/javascript">
        toastr.success('@TempData["success"]');
    </script>
}

@if (TempData["error"] != null)
{
    <script type="text/javascript">
        toastr.error('@TempData["error"]');
    </script>
}
```

Layout'a Dahil Etme

```
<!-- Views/Shared/_Layout.cshtml -->
<div class="container">
    <partial name="_Notification" />
    @RenderBody()
</div>
```

Partial Layout'a eklendiği için tüm sayfalar otomatik olarak bildirim gösterimini destekler.

15.4 Toastr Bildirim Türleri

```
toastr.success('İşlem başarılı.');
```

```
toastr.error('Bir hata oluştu.');
```

```
toastr.warning('Dikkat.');
```

```
toastr.info('Bilgi mesajı.');
```

15.5 TempData Yaşam Süresi

```
sequenceDiagram
    participant U as Kullanıcı
    participant C as Controller (POST)
    participant L as Index (GET)

    U->>C: Form submit
    C->>C: TempData["success"] = "...
    C->>L: RedirectToAction("Index")
    L-->>U: Sayfa + toastr bildirimi
    U->>L: Sayfayı yenile
    Note over L: TempData artık boş
    L-->>U: Sayfa (bildirim yok)
```

15.6 Bölüm Özeti

- TempData, redirect sonrası tek seferlik mesaj göstermenin standart yoludur.
- `Views/Shared/_Notification.cshtml` partial view ile bildirim mantığı tek noktada toplanır.
- Toastr.js ile düz `<h2>` yerine modern, sağ üstte kaybolan bildirimler kullanılır.
- Layout'a partial eklemek tüm sayfalarda bildirim altyapısını otomatik olarak aktif eder.

16. Middleware ve Pipeline

16.1 Genel Bakış

Middleware, HTTP request/response zincirinin her bir halkasıdır. İstek geldiğinde sırayla her middleware'den geçer; cevap dönerken aynı zincirden ters yönde gider. ASP.NET Core'da pipeline `Program.cs` içinde tanımlanır.

16.2 Pipeline Yapılandırması

```
// Program.cs
var app = builder.Build();

if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Home/Error");
    app.UseHsts();
}

app.UseHttpsRedirection();
app.UseStaticFiles();
app.UseRouting();
app.UseAuthentication();
app.UseAuthorization();
app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");

app.Run();
```

16.3 Sıralama Önemi

Middleware sırası, davranışı doğrudan etkiler:

```
// YANLIŞ - kim olduğu bilinmeden yetki kontrol edilemez
app.UseAuthorization();
app.UseAuthentication();

// DOĞRU
app.UseAuthentication();
app.UseAuthorization();
```

16.4 Yaygın Middleware'ler

Middleware	Görev
<code>UseHttpsRedirection</code>	HTTP isteklerini HTTPS'e yönlendirir
<code>UseStaticFiles</code>	<code>wwwroot</code> altındaki dosyaları sunar
<code>UseRouting</code>	URL'i route pattern'ine eşler
<code>UseAuthentication</code>	Kullanıcı kimliğini belirler
<code>UseAuthorization</code>	Yetkilendirme kontrolü yapar
<code>UseExceptionHandler</code>	Yakalanmamış hataları belirtilen sayfaya yönlendirir
<code>UseHsts</code>	Tarayıcıya zorunlu HTTPS bildirir
<code>MapControllerRoute</code>	İsteği controller'a iletir

16.5 Özel Middleware Yazma

```
app.Use(async (context, next) =>
{
    Console.WriteLine($"Request: {context.Request.Method} {context.Request.Path}");
    await next();
    Console.WriteLine($"Response: {context.Response.StatusCode}");
});
```

`await next()` çağrılmazsa zincir o noktada biter; bundan sonraki middleware'ler çalışmaz.

16.6 `app.Use` VS `app.Run` VS `app.Map`

Metot	Davranış
<code>app.Use(...)</code>	Sonraki middleware'e geçer (next çağrılırsa)
<code>app.Run(...)</code>	Pipeline'ı sonlandırır; next'i çağırılmaz
<code>app.Map(prefix, ...)</code>	Belirli bir URL ön eki için ayrı pipeline tanımlar

16.7 Pipeline Akışı

```
sequenceDiagram
    participant K as İstemci
    participant M1 as HTTPS Redirection
    participant M2 as Static Files
    participant M3 as Routing
    participant M4 as Authentication
    participant M5 as Authorization
    participant C as Controller

    K->>M1: Request
    M1->>M2: next()
    M2->>M3: next()
    M3->>M4: next()
    M4->>M5: next()
    M5->>C: Controller çalışır
    C-->>M5: Response
    M5-->>M4: ...
    M4-->>M3: ...
    M3-->>M2: ...
    M2-->>M1: ...
    M1-->>K: Response
```

16.8 Bölüm Özeti

- Middleware, isteğin ve cevabın her aşamada işlenebileceği bir zincir oluşturur.
- Sıralama davranışı belirler: `Authentication` her zaman `Authorization` 'dan önce gelmelidir.
- `app.Use` zinciri sürdürür; `app.Run` sonlandırır; `app.Map` dallandırır.
- Özel middleware yazılarak loglama, dil seçimi, performans ölçümü gibi cross-cutting concerns merkezileştirilebilir.

17. DTO ve ViewModel

17.1 Genel Bakış

Entity sınıfları veritabanı tablosunu birebir yansıtır; bu, View'a doğrudan gönderildiğinde hassas alanların sızmasına veya birden fazla model gerektiren ekranlarda kod kirliliğine yol açar. DTO ve ViewModel bu sorunları çözer.

17.2 DTO (Data Transfer Object)

DTO, katmanlar arası veri taşımak için kullanılan, **sadece gerekli alanları** içeren basit nesnedir.

```
// DTOs/DoktorDTO.cs
public class DoktorDTO
{
    public int Id { get; set; }
    public string AdSoyad { get; set; }
    public string BransAdi { get; set; }
}
```

Kullanım:

```
public IActionResult Index()
{
    var doktorlar = _context.Doktorlar
        .Include(d => d.Brans)
        .Select(d => new DoktorDTO
        {
            Id = d.Id,
            AdSoyad = d.AdSoyad,
            BransAdi = d.Brans.BransAdi
        })
        .ToList();

    return View(doktorlar);
}
```

Veritabanındaki `TcKimlik`, `Sifre`, `Maas` gibi hassas alanlar DTO içinde yer almaz; dolayısıyla View'a hiç ulaşmaz.

17.3 ViewModel

ViewModel, bir ekranın ihtiyaç duyduğu **tüm verileri tek bir nesnede** birleştirir. Form alanları + dropdown listeleri + ek bilgiler bir arada tutulur.

```
// ViewModels/RandevuViewModel.cs
public class RandevuViewModel
{
    public Randevu Randevu { get; set; }
    public IEnumerable<SelectListItem> BransListesi { get; set; }
    public IEnumerable<SelectListItem> DoktorListesi { get; set; }
    public IEnumerable<SelectListItem> HastaListesi { get; set; }
}
```

Controller:

```

public IActionResult Olustur()
{
    var vm = new RandevuViewModel
    {
        Randevu = new Randevu(),
        BransListesi = _context.Branslar
            .Select(b => new SelectListItem
            {
                Value = b.Id.ToString(),
                Text = b.BransAdi
            }),
        HastaListesi = _context.Hastalar
            .Select(h => new SelectListItem
            {
                Value = h.Id.ToString(),
                Text = h.AdSoyad
            })
    };
    return View(vm);
}

```

View:

```

@model RandevuViewModel

<form method="post">
    <select asp-for="Randevu.BransId"
        asp-items="Model.BransListesi">
        <option value="">-- Branş Seçin --</option>
    </select>

    <select asp-for="Randevu.HastaId"
        asp-items="Model.HastaListesi">
        <option value="">-- Hasta Seçin --</option>
    </select>

    <button type="submit">Kaydet</button>
</form>

```

17.4 Karşılaştırma

Amaç	Veri taşıma, hassas alanları gizleme	Tek View için kapsamlı veri
Kullanım yeri	Katmanlar arası, API yanıtı	Sadece View
Tipik içerik	Modelin alt kümesi	Birden fazla modelin birleşimi

17.5 Bölüm Özeti

- DTO, hassas alanları gizlemenin ve API yanıtını şekillendirmenin standart yoludur.
- ViewModel, View'a özel bir konteynerdir; form + dropdown gibi farklı veri parçalarını bir arada taşır.
- Her ikisi de Entity'leri View'a doğrudan vermenin yerine geçer ve katmanlar arası temizliği artırır.

18. AJAX

18.1 Genel Bakış

AJAX (Asynchronous JavaScript and XML), sayfa yenilemeden sunucuyla veri alışverişi yapmaya yarayan tekniktir. ASP.NET Core MVC'de bir Action `JsonResult` döndürerek, jQuery `$.ajax` ile bu veri tüketilir.

18.2 Senaryo: Branş Seçince Doktorların Yüklenmesi

Kullanıcı branş seçer; sayfanın geri kalanı yeniden yüklenmeden ilgili doktorlar dropdown'a doldurulur.

18.3 Controller Tarafı

```
// Controllers/RandevuController.cs
[HttpGet]
public JsonResult BransaGoreDoktorGetir(int bransId)
{
    var doktorlar = _context.Doktorlar
        .Where(d => d.BransId == bransId)
        .Select(d => new { d.Id, d.AdSoyad })
        .ToList();

    return Json(doktorlar);
}
```

İstek `GET /Randevu/BransaGoreDoktorGetir?bransId=2` şeklindedir; yanıt JSON olarak döner.

Beklenen yanıt:

```
[  
  { "id": 5, "adSoyad": "Dr. Ali Yılmaz" },  
  { "id": 9, "adSoyad": "Dr. Ayşe Kaya" }  
]
```

18.4 View Tarafı

```

<!-- Views/Randevu/Olustur.cshtml -->
<select id="BransId" asp-for="Randevu.BransId"
        asp-items="Model.BransListesi" class="form-control">
    <option value="">-- Branş Seçiniz --</option>
</select>

<select id="DoktorId" asp-for="Randevu.DoktorId" class="form-control">
    <option value="">-- Önce Branş Seçiniz --</option>
</select>

@section Scripts {
    <script>
        $(document).ready(function () {
            $("#BransId").change(function () {
                var bransId = $(this).val();

                $.ajax({
                    url: '/Randevu/BransaGoreDoktorGetir',
                    type: 'GET',
                    data: { bransId: bransId },
                    success: function (data) {
                        var doktorKutusu = $("#DoktorId");
                        doktorKutusu.empty();
                        doktorKutusu.append(
                            '<option value="">-- Doktor Seçiniz --</option>'
                        );
                        $.each(data, function (i, d) {
                            doktorKutusu.append(
                                '<option value="' + d.id + '">' + d.adSoyad + '</option>'
                            );
                        });
                    },
                    error: function () {
                        alert('Doktorlar getirilirken hata oluştu.');
```

18.5 Akış Diyagramı

```
sequenceDiagram
    participant K as Kullanıcı
    participant V as View (Browser)
    participant C as RandevuController

    K->>V: Branş seçer (change event)
    V->>C: GET /Randevu/BransaGoreDoktorGetir?bransId=2
    C->>C: DB sorgula
    C-->>V: JSON doktor listesi
    V->>V: Dropdown'ı doldur
    V-->>K: Yenilenmiş dropdown (sayfa yenilenmedi)
```

18.6 Bölüm Özeti

- AJAX, sayfa yenilemeden sunucu ile veri alışverişine olanak tanır.
- Controller tarafında `JsonResult` veya `ActionResult + return Json(...)` ile JSON döndürülür.
- jQuery `$.ajax` 'in `success` callback'i içinde DOM güncellenir.
- Cascading dropdown (branş → doktor) en yaygın AJAX senaryolarından biridir.

19. Kategori CRUD Modülü

19.1 Amaç

Kullanıcı, sistemdeki kategorileri listeleyebilir, yeni kategori ekleyebilir, mevcut kategoriye düzenleyebilir ve silebilir. Veriler SQL Server'da Code First ile oluşturulan `Categories` tablosunda tutulur.

19.2 Akış

```
flowchart LR
    A[Index - Listele] --> B[Create - Ekle]
    A --> C[Edit - Düzenle]
    A --> D[Delete - Sil]
    B --> A
    C --> A
    D --> A
```

19.3 Model

```
// Models/Category.cs
using System.ComponentModel;
using System.ComponentModel.DataAnnotations;

public class Category
{
    [Key]
    public int Id { get; set; }

    [Required]
    [MaxLength(30)]
    public string Name { get; set; }

    [DisplayName("Display Order")]
    [Range(1, 100, ErrorMessage = "Display Order must be between 1 and 100.")]
    public int DisplayOrder { get; set; }

    public DateTime CreatedDateTime { get; set; } = DateTime.Now;
}
```

19.4 DbContext

```
// Data/ApplicationDbContext.cs
public class ApplicationDbContext : DbContext
{
    public ApplicationDbContext(DbContextOptions<ApplicationDbContext> options)
        : base(options) { }

    public DbSet<Category> Categories { get; set; }
}
```

19.5 Controller

```
// Controllers/CategoryController.cs
public class CategoryController : Controller
{
    private readonly ApplicationDbContext _db;

    public CategoryController(ApplicationDbContext db)
    {
        _db = db;
    }

    // INDEX
    public IActionResult Index()
    {
        IEnumerable<Category> objCategoryList = _db.Categories;
        return View(objCategoryList);
    }

    // CREATE - GET
    public IActionResult Create()
    {
        return View();
    }

    // CREATE - POST
    [HttpPost]
    [ValidateAntiForgeryToken]
    public IActionResult Create(Category obj)
    {
        if (obj.Name == obj.DisplayOrder.ToString())
        {
            ModelState.AddModelError("name",
                "The DisplayOrder cannot exactly match the Name.");
        }

        if (ModelState.IsValid)
        {
            _db.Categories.Add(obj);
            _db.SaveChanges();
            TempData["success"] = "Category created successfully";
            return RedirectToAction("Index");
        }
        return View(obj);
    }
}
```

```

// EDIT - GET
public IActionResult Edit(int? id)
{
    if (id == null || id == 0)
        return NotFound();

    var categoryFromDb = _db.Categories.Find(id);
    if (categoryFromDb == null)
        return NotFound();

    return View(categoryFromDb);
}

// EDIT - POST
[HttpPost]
[ValidateAntiForgeryToken]
public IActionResult Edit(Category obj)
{
    if (ModelState.IsValid)
    {
        _db.Categories.Update(obj);
        _db.SaveChanges();
        TempData["success"] = "Category updated successfully";
        return RedirectToAction("Index");
    }
    return View(obj);
}

// DELETE - GET
public IActionResult Delete(int? id)
{
    if (id == null || id == 0)
        return NotFound();

    var categoryFromDb = _db.Categories.Find(id);
    if (categoryFromDb == null)
        return NotFound();

    return View(categoryFromDb);
}

// DELETE - POST
[HttpPost, ActionName("Delete")]
[ValidateAntiForgeryToken]

```

```
public IActionResult DeletePOST(int? id)
{
    var obj = _db.Categories.Find(id);
    if (obj == null)
        return NotFound();

    _db.Categories.Remove(obj);
    _db.SaveChanges();
    TempData["success"] = "Category deleted successfully";
    return RedirectToAction("Index");
}
}
```

19.6 View'lar

Index.cshtml

```

@model IEnumerable<Category>

<div class="container p-3">
    <div class="row pt-4">
        <div class="col-6">
            <h2 class="text-primary">Category List</h2>
        </div>
        <div class="col-6 text-end">
            <a asp-controller="Category" asp-action="Create"
              class="btn btn-primary">
                <i class="bi bi-plus-circle"></i> Create New Category
            </a>
        </div>
    </div>

    <table class="table table-bordered table-striped" style="width:100%">
        <thead>
            <tr>
                <th>Name</th>
                <th>Display Order</th>
                <th>Actions</th>
            </tr>
        </thead>
        <tbody>
            @foreach (var obj in Model)
            {
                <tr>
                    <td style="width:50%">@obj.Name</td>
                    <td style="width:30%">@obj.DisplayOrder</td>
                    <td>
                        <div class="w-75 btn-group" role="group">
                            <a asp-controller="Category" asp-action="Edit"
                              asp-route-id="@obj.Id"
                              class="btn btn-primary mx-2">
                                <i class="bi bi-pencil-square"></i> Edit
                            </a>
                            <a asp-controller="Category" asp-action="Delete"
                              asp-route-id="@obj.Id"
                              class="btn btn-danger mx-2">
                                <i class="bi bi-trash"></i> Delete
                            </a>
                        </div>
                    </td>
                </tr>
            }
        </tbody>
    </table>

```

```

        }
    </tbody>
</table>
</div>

```

Create.cshtml

```

@model Category

<form method="post" asp-action="Create">
    <div class="border p-3 mt-4">
        <div class="row pb-2">
            <h2 class="text-primary">Create Category</h2>
            <hr />
        </div>

        <div asp-validation-summary="ModelOnly" class="text-danger"></div>

        <div class="mb-3">
            <label asp-for="Name"></label>
            <input asp-for="Name" class="form-control" />
            <span asp-validation-for="Name" class="text-danger"></span>
        </div>

        <div class="mb-3">
            <label asp-for="DisplayOrder"></label>
            <input asp-for="DisplayOrder" class="form-control" />
            <span asp-validation-for="DisplayOrder" class="text-danger"></span>
        </div>

        <button type="submit" class="btn btn-primary"
            style="width:150px">Create</button>
        <a asp-controller="Category" asp-action="Index"
            class="btn btn-secondary"
            style="width:150px">Back to List</a>
    </div>
</form>

@section Scripts {
    <partial name="_ValidationScriptsPartial" />
}

```

Edit.cshtml

```

@model Category

<form method="post" asp-action="Edit">
    <div class="border p-3 mt-4">
        <div class="row pb-2">
            <h2 class="text-primary">Edit Category</h2>
            <hr />
        </div>

        <input type="hidden" asp-for="Id" />

        <div class="mb-3">
            <label asp-for="Name"></label>
            <input asp-for="Name" class="form-control" />
            <span asp-validation-for="Name" class="text-danger"></span>
        </div>

        <div class="mb-3">
            <label asp-for="DisplayOrder"></label>
            <input asp-for="DisplayOrder" class="form-control" />
            <span asp-validation-for="DisplayOrder" class="text-danger"></span>
        </div>

        <button type="submit" class="btn btn-primary"
            style="width:150px">Update</button>
        <a asp-controller="Category" asp-action="Index"
            class="btn btn-secondary"
            style="width:150px">Back to List</a>
    </div>
</form>

@section Scripts {
    <partial name="_ValidationScriptsPartial" />
}

```

Delete.cshtml

```

@model Category

<form method="post" asp-action="Delete">
    <div class="border p-3 mt-4">
        <div class="row pb-2">
            <h2 class="text-danger">Delete Category</h2>
            <hr />
        </div>

        <input type="hidden" asp-for="Id" />

        <div class="mb-3">
            <label asp-for="Name"></label>
            <input asp-for="Name" class="form-control" disabled />
        </div>

        <div class="mb-3">
            <label asp-for="DisplayOrder"></label>
            <input asp-for="DisplayOrder" class="form-control" disabled />
        </div>

        <button type="submit" class="btn btn-danger"
            style="width:150px">Delete</button>
        <a asp-controller="Category" asp-action="Index"
            class="btn btn-secondary"
            style="width:150px">Back to List</a>
    </div>
</form>

```

19.7 Beklenen Sonuçlar

Senaryo	Sonuç
Geçerli form ile Create	Kayıt eklenir, Index'e dönülür, toastr "created" gösterilir.
Boş form ile Create	<code>ModelState.IsValid</code> false; aynı sayfada inline + summary hataları görünür.
Var olan ID ile Edit	Form veriyle dolu açılır; Update sonrası kayıt güncellenir.
Geçersiz ID ile Edit	<code>NotFound()</code> döner (HTTP 404).
Delete onayı	Kayıt silinir, toastr "deleted" gösterilir.

19.8 Notlar

- POST tarafındaki Delete metodu C# imzasında `Delete` adıyla aynı olamaz; bu yüzden `[HttpPost, ActionName("Delete")]` ile route ismi sabit tutulur, C# metodu `DeletePOST` adıyla yazılır.
- Update için `_db.Categories.Update(obj)` primary key'e bakarak doğru satırı bulur; değişen alanları tek tek belirtmeye gerek yoktur.
- Custom validation (Name == DisplayOrder kontrolü) `ModelState.AddModelError` ile manuel eklenir.

20. Hastane Randevu Modülü

20.1 Amaç

Hasta, branş ve doktor verisi üzerinden randevu oluşturulan ve listelenen bir modül. Branş seçimi yapıldığında ilgili doktorlar AJAX ile dinamik olarak yüklenir. Veritabanı Database First yaklaşımıyla, var olan SQL Server şemasından scaffold edilir.

20.2 Akış

flowchart TB

A[Branş Tablosu] --> B[Doktor Tablosu]

A --> R[Randevu Tablosu]

B --> R

H[Hasta Tablosu] --> R

20.3 Modeller

```
// Models/Brans.cs
public class Brans
{
    public int Id { get; set; }
    public string BransAdi { get; set; }
    public bool IsDeleted { get; set; }
    public DateTime? DeleteTime { get; set; }

    public List<Doktor> Doktorlar { get; set; }
}

// Models/Doktor.cs
public class Doktor
{
    public int Id { get; set; }
    public string AdSoyad { get; set; }

    public int BransId { get; set; }
    public Brans Brans { get; set; }

    public List<Randevu> Randevular { get; set; }
}

// Models/Hasta.cs
public class Hasta
{
    public int Id { get; set; }
    public string AdSoyad { get; set; }
    public string TcKimlik { get; set; }
    public DateTime DogumTarihi { get; set; }

    public List<Randevu> Randevular { get; set; }
}

// Models/Randevu.cs
public class Randevu
{
    public int Id { get; set; }

    public int HastaId { get; set; }
    public Hasta Hasta { get; set; }

    public int DoktorId { get; set; }
    public Doktor Doktor { get; set; }
```

```

    public int BransId { get; set; }
    public Brans Brans { get; set; }

    public DateTime Tarih { get; set; }
}

```

20.4 DbContext (Database First Üretimi)

`Scaffold-DbContext` komutu, var olan veritabanından modelleri ve DbContext'i üretir.

```

Scaffold-DbContext "YOUR_CONNECTION_STRING"
    Microsoft.EntityFrameworkCore.SqlServer
    -OutputDir Models

```

Üretilen DbContext örnek:

```

// Data/HastaneDbContext.cs
public class HastaneDbContext : DbContext
{
    public HastaneDbContext(DbContextOptions<HastaneDbContext> options)
        : base(options) { }

    public DbSet<Hasta> Hastalar { get; set; }
    public DbSet<Doktor> Doktorlar { get; set; }
    public DbSet<Brans> Branslar { get; set; }
    public DbSet<Randevu> Randevular { get; set; }
}

```

20.5 Program.cs

```

builder.Services.AddDbContext<HastaneDbContext>(options =>
    options.UseSqlServer(
        builder.Configuration.GetConnectionString("HastaneCon")
    )
);

```

`appsettings.json` :

```

{
  "ConnectionStrings": {
    "HastaneCon": "YOUR_CONNECTION_STRING"
  }
}

```

20.6 Branş Silme (Soft Delete)

Branş kaydı veritabanından gerçekten silinmez; `IsDeleted` ve `DeleteTime` alanları işaretlenir. Bu, "soft delete" desenidir.

```
// Controllers/BransController.cs
[HttpGet]
public IActionResult Sil(int id)
{
    var brans = _context.Branslar.Find(id);
    if (brans == null)
        return NotFound();

    var dto = new BransModelDTO
    {
        Id = brans.Id,
        BransAdi = brans.BransAdi
    };

    return View(dto);
}

[HttpPost, ActionName("Sil")]
public IActionResult SilOnaylandi(int id)
{
    var brans = _context.Branslar.Find(id);
    if (brans != null)
    {
        brans.DeleteTime = DateTime.Now;
        brans.IsDeleted = true;
        _context.SaveChanges();
    }
    return RedirectToAction("Index");
}
```

Aktif kayıtları listelerken filtrelemek gerekir:

```
public IActionResult Index()
{
    var aktif = _context.Branslar
        .Where(b => !b.IsDeleted)
        .ToList();
    return View(aktif);
}
```

20.7 Randevu Oluşturma (AJAX ile Cascading Dropdown)

ViewModel

```
// ViewModels/RandevuViewModel.cs
public class RandevuViewModel
{
    public Randevu Randevu { get; set; }
    public IEnumerable<SelectListItem> BransListesi { get; set; }
    public IEnumerable<SelectListItem> HastaListesi { get; set; }
}
```

Controller

```
// Controllers/RandevuController.cs
public class RandevuController : Controller
{
    private readonly HastaneDbContext _context;

    public RandevuController(HastaneDbContext context)
    {
        _context = context;
    }

    public IActionResult Index()
    {
        var randevular = _context.Randevular
            .Include(r => r.Hasta)
            .Include(r => r.Doktor)
            .ThenInclude(d => d.Brans)
            .OrderBy(r => r.Tarih)
            .ToList();
        return View(randevular);
    }

    public IActionResult Olustur()
    {
        var vm = new RandevuViewModel
        {
            Randevu = new Randevu(),
            BransListesi = _context.Branslar
                .Where(b => !b.IsDeleted)
                .Select(b => new SelectListItem
                {
                    Value = b.Id.ToString(),
                    Text = b.BransAdi
                }),
            HastaListesi = _context.Hastalar
                .Select(h => new SelectListItem
                {
                    Value = h.Id.ToString(),
                    Text = h.AdSoyad
                })
        };
        return View(vm);
    }

    [HttpPost]
```

```
[ValidateAntiForgeryToken]
public IActionResult Olustur(RandevuViewModel vm)
{
    if (ModelState.IsValid)
    {
        _context.Randevular.Add(vm.Randevu);
        _context.SaveChanges();
        TempData["success"] = "Randevu oluşturuldu";
        return RedirectToAction("Index");
    }
    return View(vm);
}

[HttpGet]
public JsonResult BransaGoreDoktorGetir(int bransId)
{
    var doktorlar = _context.Doktorlar
        .Where(d => d.BransId == bransId)
        .Select(d => new { d.Id, d.AdSoyad })
        .ToList();

    return Json(doktorlar);
}
}
```

View

```

@model RandevuViewModel

<form method="post" asp-action="Olustur">
    <div class="mb-3">
        <label asp-for="Randevu.HastaId">Hasta</label>
        <select asp-for="Randevu.HastaId"
            asp-items="Model.HastaListesi"
            class="form-control">
            <option value="">-- Hasta Seçiniz --</option>
        </select>
    </div>

    <div class="mb-3">
        <label asp-for="Randevu.BransId">Brans</label>
        <select id="BransId" asp-for="Randevu.BransId"
            asp-items="Model.BransListesi"
            class="form-control">
            <option value="">-- Brans Seçiniz --</option>
        </select>
    </div>

    <div class="mb-3">
        <label asp-for="Randevu.DoktorId">Doktor</label>
        <select id="DoktorId" asp-for="Randevu.DoktorId" class="form-control">
            <option value="">-- Önce Brans Seçiniz --</option>
        </select>
    </div>

    <div class="mb-3">
        <label asp-for="Randevu.Tarih">Tarih</label>
        <input asp-for="Randevu.Tarih" type="datetime-local" class="form-control" />
    </div>

    <button type="submit" class="btn btn-primary">Kaydet</button>
</form>

@section Scripts {
    <script>
        $(document).ready(function () {
            $("#BransId").change(function () {
                var bransId = $(this).val();

                $.ajax({
                    url: '/Randevu/BransaGoreDoktorGetir',

```

```

        type: 'GET',
        data: { bransId: bransId },
        success: function (data) {
            var doktorKutusu = $("#DoktorId");
            doktorKutusu.empty();
            doktorKutusu.append(
                '<option value="">-- Doktor Seçiniz --</option>'
            );
            $.each(data, function (i, d) {
                doktorKutusu.append(
                    '<option value="' + d.id + '">' + d.adSoyad + '</option>'
                );
            });
        },
        error: function () {
            alert('Doktorlar getirilirken hata oluştu.');
```

20.8 Beklenen Sonuçlar

Senaryo	Sonuç
Branş seçilir	AJAX çağırısı yapılır; doktor dropdown'ı dinamik dolar.
Geçerli form gönderilir	Randevu eklenir, Index sayfasına yönlendirilir, toastr "oluşturuldu" bildirimi görünür.
Doktor seçilmeden gönderilir	ModelState geçersiz; form aynı veriyle yeniden gösterilir.
Branş silinir	<code>IsDeleted = true</code> yapılır; aktif listeden düşer ancak tarihçesi korunur.

20.9 Notlar

- Soft delete deseni, "yanlışlıkla sildim" durumlarına karşı geri alma imkanı sağlar ve geçmiş raporları korur.
- AJAX endpoint'i sadece gerekli alanları (`Id` , `AdSoyad`) döndürür; tam Doktor nesnesini taşımamak performans ve güvenlik açısından önemlidir.
- Randevu listesinde `Include` + `ThenInclude` ile üç tablo birden tek sorguda yüklenir; aksi halde lazy loading veya N+1 sorun olur.

Ekler

Ek A — NuGet Paket Yönetimi Komutları

Package Manager Console (PMC):

```
Install-Package Microsoft.EntityFrameworkCore
Install-Package Microsoft.EntityFrameworkCore.SqlServer
Install-Package Microsoft.EntityFrameworkCore.Tools

Add-Migration "InitialCreate"
Update-Database

Scaffold-DbContext "YOUR_CONNECTION_STRING" Microsoft.EntityFrameworkCore.SqlServer -
OutputDir Models
```

Ek B — Bootswatch Tema Kurulumu

1. `bootswatch.com` adresinden istenen temanın `bootstrap.min.css` dosyası indirilir.
2. `wwwroot/css/bootswatch-theme.css` olarak kaydedilir.
3. `_Layout.cshtml` içinde varsayılan Bootstrap link'i kaldırılıp şu eklenir:

```
<link rel="stylesheet" href="~/css/bootswatch-theme.css" />
```

1. Bootstrap 5 JS bundle linki güncellenir.

Ek C — Bootstrap Icons

```
<link rel="stylesheet"
      href="https://cdn.jsdelivr.net/npm/bootstrap-icons/font/bootstrap-icons.css" />

<i class="bi bi-plus-circle"></i>
<i class="bi bi-pencil-square"></i>
<i class="bi bi-trash"></i>
```

Ek D — Razor Runtime Compilation

View değişikliklerinin uygulama yeniden başlatılmadan görünmesi için:

```
builder.Services.AddControllersWithViews()
                .AddRazorRuntimeCompilation();
```

Gerekli paket: `Microsoft.AspNetCore.Mvc.Razor.RuntimeCompilation`.